

عربي وأسئلة متوقعة بإجاباتها - DocuMind AI

شرح

دليل عربي للتحضير للعرض والمناقشة - معمارية، APIs، Agent، وأسئلة متوقعة

هدف الملف

وAPIs، مع التركيز على جزء الإيجنت، منع الهلوسة، الاعتماد على مصادر PDF فقط، وتوليد الامتحانات من نفس المصدر. ليست فقط شرح ما يفعله المشروع، بل توضيح كيف تم تصميمه كمنظومة إنتاجية قابلة للربط مع Backend و Frontend هذا الملف مخصص للتحضير للمناقشة والعرض أمام لجنة التحكيم. الفكرة

1. شرح مختصر للمشروع

يجب فقط من المعلومات الموجودة داخل الملفات المرفوعة PDF هو نظام ذكاء اصطناعي للتعامل مع ملفات DocuMind AI آمنة ومبنية على المصدر. المستخدم يرفع ملف أو أكثر داخل شات، ثم يسأل أسئلة طبيعية بالعربية أو الإنجليزية، والنظام بطريقة.

لا يخترع إجابة، بل يرد بشكل مهني بأن المصدر غير كاف، ويوجه المستخدم لاستشارة طبيب أو متخصص حسب المجال. أن ال LLM "يعرف" الإجابة من ذاكرته العامة. المصدر الحقيقي هو ال PDF. لذلك لو السؤال غير مدعوم من الملف، النظام المشروع لا يعتمد على

2. وصف سريع للعرض

نفس النظام يدعم توليد امتحانات من مصدر الدكتور أو المحاضر مع تحديد عدد الأسئلة والصعوبة ونوع الأسئلة واللغة. يرسل هذه الأدلة فقط لل LLM، ثم يتحقق من دعم الإجابة بالمصدر وبحسب hallucination risk و groundedness score. PDF، والباك إند يبنى indexes للبحث داخلها، ثم الإيجنت يعيد صياغة السؤال، يبحث في المصدر، يسترجع الأدلة المناسبة، المشروع عبارة عن Adaptive Multimodal RAG Platform. المستخدم يرفع ملفات

3. الفكرة الأساسية

الفكرة الأساسية هي:

- المصدر المرفوع هو الحقيقة.
- ال LLM ليس مصدر الحقيقة.
- أي إجابة يجب أن تكون مدعومة بصفحات أو أدلة من ال PDF.
- لو الدليل غير موجود، النظام يرفض الإجابة بأدب.
- ال API يرجع response منظم يصلح للفرونت أو لأي نظام خارجي.

4. المعمارية العامة

```

User
|
v
React Frontend
|
| HTTP / JSON / PDF Upload
v
FastAPI Backend
|
+--> Auth Layer
|     Signup / Login / JWT
|
+--> Document APIs
|     Upload PDFs / Persist Sources / Build Indexes
|
+--> Agent Layer
|     Memory / Query Rewrite / Retrieval / Generation / Verification
|
+--> Database Layer
|     Users / Uploaded PDF Sources
|
v
PostgreSQL + Runtime FAISS/BM25 Indexes

```

5. ما هو الإيجنت في المشروع؟

prompt ولا مجرد chatbot. الإيجنت هو طبقة orchestration داخل الباك إند. دوره أن يدير خطوات الذكاء الاصطناعي كاملة: الإيجنت في هذا المشروع ليس مجرد

- يفصل كل شات عن الآخر.
- يستخدم ذاكرة قصيرة للمحادثة لفهم الإشارات مثل "اشرح النقطة السابقة".
- لا يعتبر الذاكرة مصدرا للمعلومة.
- يبنى أو يختار الـ index المناسب للشات الحالي.
- يعيد صياغة السؤال ليصبح أفضل للبحث.
- يسترجع الأدلة من ملفات PDF.
- يرسل الأدلة فقط للـ LLM.
- يستخرج claims من الإجابة.
- يتحقق من دعم الإجابة بالمصدر.
- يحسب hallucination risk.
- يرفض الإجابة لو الدليل غير كاف.
- يولد امتحانات من نفس المصدر.

الإيجنت موجود عمليا في:

- ChatRagManager
 - RagService
- RAG pipeline modules

6. مسار عمل الإيجنت

User Question

```

|
v
ChatRagManager
|
+--> [REDACTED] memory [REDACTED] [REDACTED] [REDACTED] [REDACTED]
|
v
RagService.ask()
|
+--> [REDACTED] [REDACTED] index [REDACTED] [REDACTED]
|
v
RAG Pipeline
|
+--> Language Detection
+--> Query Classification
+--> Query Rewriting
+--> Hybrid Retrieval
+--> Reranking
+--> Context Building
+--> LLM Generation
+--> Claim Extraction
+--> Citation Verification
|
v
Safety Layer
|
+--> [REDACTED] [REDACTED] [REDACTED] [REDACTED]: Refusal
+--> [REDACTED] [REDACTED] [REDACTED] [REDACTED]: Refusal
|
v
Structured API Response

```

7. لماذا المشروع ليس RAG عادي؟

أي RAG بسيط يمكنه البحث في chunks ثم إرسالها للـ LLM. لكن DocuMind AI يضيف:

- Query rewriting.
- Hybrid retrieval بين semantic search و BM25.
- Chat-scoped source isolation.
- محدودة لكل شات Memory.
- Source-only refusal.
- Hallucination risk score.
- Groundedness score.
- Claims and citation verification.
- Persistent uploaded PDF sources.
- Exam generation API.
- منظمة للفرونت والباك Pydantic schemas.
- Production deployment باستخدام Docker و Railway.

8. تصميم الـ APIs

و response schema واضح. هذا مهم لأن المشروع سيتم ربطه بفرونٲ إنڊ وباك إنڊ وقد يستخدمه أي client خارجي. الباك إنڊ مبني باستخدام FastAPI. كل endpoint له request المكونات:

- Controllers: تستقبل HTTP requests.
- Schemas: تعرف شكل البيانات باستخدام
- Services: تحتوي business logic.
- Repositories: تتعامل مع قاعدة البيانات.
- Models: تمثل الجداول.
- RAG modules: تحتوي منطق الذكاء الاصطناعي.

9. أهم APIs في المشروع

Authentication APIs

الغرض منها إنشاء حسابات وتسجيل دخول وحماية البيانات.

Endpoints متوقعة:

- POST /auth/signup
- POST /auth/login
- GET /auth/me

الأهمية:

- كل مستخدم له مصادره الخاصة.
- لا يجب أن يرى مستخدم ملفات مستخدم آخر.
- ال JWT يستخدم لحماية endpoints.

Document Upload API

Endpoint:

POST /chats/{chat_id}/sources

دوره:

- يستقبل ملفات PDF.
- يتحقق أن الملفات PDF فقط.
- يحفظ الملفات مؤقتا للمعالجة.
- يبني knowledge base و indexes.
- يخزن نسخة من ال PDF في قاعدة البيانات.
- يرجع حالة البناء وعدد ال chunks.

Ask API

Endpoint:

POST /ask

Request:

```
{
  "query": "الذكارة تستخدم فقط لفهم السياق، وليست مصدر معلومة.",
  "chat_id": "chat-123",
  "document_ids": null,
  "verbose": false,
  "use_memory": true
}
```

Response يحتوي:

- answer
- query metadata
- confidence
- hallucination_risk
- groundedness_score
- unsupported_claims
 - sources
 - evidence
 - claims
- raw debug data

Chat Memory API

Endpoint:

GET /chats/{chat_id}/memory

دوره عرض الذاكرة المحدودة للشات. الذاكرة تستخدم فقط لفهم السياق، وليست مصدر معلومة.

Exam API

Endpoint:

POST /exam

Request:

```
{
  "chat_id": "chat-123",
  "topic": "Chapter 2",
  "difficulty": "medium",
  "question_count": 10,
  "total_marks": 100,
  "question_types": ["mcq", "short_answer", "essay"],
  "language": "ar"
}
```

دوره توليد امتحان من المصدر المرفوع فقط مع إجابات وشرح وصفحات المصدر.

10. Query Rewriting

الـ PDF. أحيانا سؤال المستخدم يكون قصيرا أو عاميا أو يشير لصفحة أو جدول. النظام يحوله إلى query أوضح للـ retriever. إعادة صياغة السؤال هدفها تحسين البحث داخل

مثال:

السؤال الأصلي:

5 page 5 table

الصياغة المحسنة للبحث:

5 page 5 table

الفائدة:

- تحسين retrieval.
- دعم أفضل للعربية والإنجليزية.
- فهم page filters.
- تحسين البحث في الجداول والرسوم والملخصات.

11. سياسة المصدر فقط

النظام لا يجب أن يجيب من خارج ملفات PDF المرفوعة.

لو لا يوجد دليل كاف، تكون الإجابة:

النتيجة هي: لا يوجد دليل كاف. لا يمكن الإجابة على هذا السؤال بناءً على المعلومات المتاحة. يرجى إعادة صياغة السؤال أو تقديم مزيد من التفاصيل.

هذه الصياغة أفضل من "لا أعرف" لأنها مهنية وتوجه المستخدم للحل الصحيح.

12. قياس الهلوسة والـ Groundedness

النظام يرجع:

- hallucination_risk
- groundedness_score
- unsupported_claims
 - claims
 - sources
 - evidence

المعنى:

- hallucination risk: احتمال وجود ادعاءات غير مدعومة بالمصدر.
- groundedness score: درجة ارتباط الإجابة بالأدلة المسترجعة.

- unsupported claims: عدد الجمل التي لم يتم دعمها بدليل كاف لو خطر الهلوسة عالي، النظام يرفض الإجابة.

13. Persistent Indexing

BM25 أنها قد تضيع بعد restart لأنها في الذاكرة. الحل في المشروع هو تخزين ملفات PDF الأصلية في قاعدة البيانات. مشكلة الـ FAISS runtime indexes المسار:

```
Upload PDF
|
+--> Build runtime index
|
+--> Save PDF bytes in database

Server Restart
|
+--> Runtime index missing
|
+--> Rebuild from persisted PDFs
|
+--> Continue answering
```

هذا مهم جدا في Railway أو أي Cloud deployment لأن السيرفر ممكن يعيد التشغيل.

14. قاعدة البيانات

قاعدة البيانات تخزن:

- المستخدمين.
 - بيانات تسجيل الدخول.
 - ملفات PDF الأصلية.
 - علاقة كل PDF بالمستخدم والنشأت.
- هذا يجعل المشروع أقرب للإنتاج الحقيقي وليس مجرد notebook demo.

15. الفرونت إند

الفرونت مبني بـ React وTypeScript ويدعم:

- Landing page.
- Login/signup.
- PDF upload.
- Chat UI.
- عرض المصادر والثقة.
- Exam generation.
- حسب دعم المتصفح Voice input.

- Responsive mobile layout.
- Dark/light mode.
- Language toggle.

16. النشر Production

المشروع مجهز للنشر على Railway باستخدام Docker.

المسار:

```

GitHub
|
v
Railway
|
v
Docker Build
|
+--> Build frontend
+--> Install backend dependencies
+--> Install runtime/OCR dependencies
+--> Run FastAPI using Uvicorn

```

أهم Variables:

- GROQ_API_KEY
- DATABASE_URL
- JWT_SECRET_KEY
- DOCUMIND_LOW_MEMORY
- DOCUMIND_MAX_ANSWER_TOKENS

17. أسئلة متوقعة وإجاباتها

سؤال 1: ما المشكلة التي يحلها المشروع؟

بدل أن يعتمد النظام على ذاكرة الـ LLM، يجبر الإجابة أن تكون مبنية على المصدر المرفوع فقط، ويرفض لو لا يوجد دليل كاف. المشروع يحل مشكلة الإجابات غير الموثوقة من ملفات PDF.

سؤال 2: ما الفرق بين مشروعكم و ChatGPT file upload؟

مشروعنا منتج قابل للربط والنشر، وليس مجرد واجهة شات. لدينا generation structured responses، APIs، authentication، database، source persistence، hallucination scoring، exam للفرونت.

سؤال 3: أين الإيجنت؟

و RagService وال RAG pipeline. هو الذي يدير الذاكرة، البحث، التوليد، التحقق، الرفض، وتوليد الامتحانات. الإيجنت هو طبقة orchestration في الباك إند، ويتمثل في ChatRagManager

سؤال 4: هل الإيجنت autonomous agent؟

يفعل أي شيء. أدواته محدودة: retrieval، query rewrite، LLM generation، verification، refusal، exam generation. هو controlled domain-specific agent وليس agent مفتوح

سؤال 5: لماذا لم تستخدموا agent مفتوح يبحث في الإنترنت؟

يحتاج أمان وثقة. في التعليم أو الطب، لا نريد إجابات من خارج مصدر الدكتور أو الملف المرفوع. لذلك اخترنا controlled agent. لأن المشروع

سؤال 6: ما أدوات الإيجنت؟

BM25، dense retrieval، query classification، language detection، query rewriting، exam JSON generation، retrieval، table retrieval، visual retrieval، reranking، context building، LLM generation، claim extraction، citation verification، hallucination scoring، refusal policy هي أدواته الداخلية

سؤال 7: هل يستخدم النظام الإنترنت؟

هو ملفات PDF المرفوعة. ال LLM يستخدم قدرته اللغوية لصياغة الإجابة، لكن factual grounding يأتي من ال PDF فقط. لا. المصدر الواقعي الوحيد

سؤال 8: كيف تمنعون الهلوسة؟

من الأدلة فقط، استخراج claims، التحقق من citations، حساب hallucination risk، ثم الرفض إذا كانت الإجابة غير مدعومة. نمنع الهلوسة بعدة طبقات: retrieval من المصدر، generation

سؤال 9: ماذا يحدث لو السؤال من خارج الملف؟

ويقول إن المصدر غير كاف. لو السؤال طبي يوصي باستشارة طبيب، ولو في مجال آخر يوصي بمتخصص في نفس المجال. النظام يرفض بلطف

سؤال 10: لماذا لا تقولون فقط "لا أعرف"؟

لأن الرد المهني يجب أن يوضح السبب ويوجه المستخدم للخطوة الصحيحة. "المصدر غير كاف" أدق من "لا أعرف".

سؤال 11: ما معنى Query Rewriting؟

هي تحويل سؤال المستخدم إلى صيغة أفضل للبحث داخل ال PDF بدون تغيير معناه.

سؤال 12: هل Query Rewriting قد تغير معنى السؤال؟

أن تغير المعنى. هي تضيف hints للبحث مثل page number أو table أو chart، مع الاحتفاظ بالسؤال الأصلي في response. لا يجب

سؤال 13: ما هو Hybrid Retrieval؟

الجمع بين keyword retrieval و semantic retrieval. الأول يفهم المعنى، والثاني يمسك الكلمات الدقيقة والأرقام والتعريفات. هو

سؤال 14: لماذا FAISS؟

الأكبر يمكن الانتقال إلى MVP و hackathon خصوصا في runtime في vector similarity سريع ومناسب للبحث FAISS Pinecone أو Qdrant أو pgvector للإنتاج

سؤال 15: هل FAISS يضيع بعد restart؟

نعم، runtime index قد يضيع. لذلك نخزن ملفات PDF الأصلية في database ونعيد بناء ال index عند الحاجة.

سؤال 16: لماذا لا تخزنون ال vector index فقط؟

PDF الأصلي يسمح بإعادة بناء الفهرس عند تغيير chunking أو embeddings أو بعد restart. مستقبلا يمكن تخزين الاثنين. تخزين

سؤال 17: ما دور PostgreSQL؟

تخزين المستخدمين ومصادر PDF المرفوعة وربط كل مصدر بال user وال chat.

سؤال 18: هل كل شات منفصل؟

نعم. كل chat له مصادره و index الخاص به حتى لا تختلط ملفات المستخدمين أو الشاتات.

سؤال 19: كيف تعمل الذاكرة؟

تخزن آخر رسائل بشكل محدود. تستخدم لفهم الإشارات فقط، مثل "اشرح النقطة السابقة"، لكنها ليست مصدرا factual. الذاكرة

سؤال 20: هل يمكن لل LLM الإجابة من الذاكرة؟

لا. الذاكرة تساعد في فهم السؤال، لكن الإجابة النهائية يجب أن تكون من ال PDF.

سؤال 21: ما هو LLM provider؟

المشروع يستخدم Groq من خلال GROQ_API_KEY كمتغير بيئة.

سؤال 22: هل Groq API key وحده يكفي؟

لا. نحتاج أيضا DATABASE_URL للبيانات و JWT_SECRET_KEY للأمان، بالإضافة إلى إعدادات التشغيل.

سؤال 23: ما أهم APIs؟

أهم APIs هي exam، chat memory، ask، upload sources، login، signup، و health.

سؤال 24: لماذا FastAPI؟

لأنه سريع، typed، مناسب للإنتاج، ويدعم Pydantic و automatic API docs.

سؤال 25: لماذا Pydantic؟

لضمان validation واضح وشكل ثابت لل request وال response، وهذا يقلل أخطاء الربط مع الفرونت.

سؤال 26: ما شكل Ask Response؟

confidence و hallucination risk و groundedness score و unsupported claims و sources و evidence و claims و raw data. يرجع answer و query metadata

سؤال 27: لماذا تعرضون hallucination risk للفرونت؟

لأن المستخدم ولجنة التقييم يجب أن يروا درجة الثقة والدعم، وليس نص الإجابة فقط.

سؤال 28: ما هو Exam API؟

وعدد الأسئلة والدرجات والأنواع واللغة و difficulty و topic المرفوع فقط، مع تحديد PDF يولد امتحانا من ال API

سؤال 29: كيف يستفيد الدكتور الجامعي؟

الدكتور lecture notes أو chapter، ثم يطلب امتحانا بمواصفات معينة. النظام يرجع أسئلة وإجابات وشرح وصفحات المصدر. يرفع

سؤال 30: كيف تضمنون أن أسئلة الامتحان من الملف؟

الامتحان يمر بنفس RAG flow، مع prompt يلزم ال LLM باستخدام المصدر فقط وإخراج JSON منظم مع صفحات المصدر. توليد

سؤال 31: ماذا لو رجع Exam API JSON غير صحيح؟

الباك إند يكتشف ذلك ويرجع error controlled بدل إرسال بيانات مكسورة للفرونت.

سؤال 32: هل النظام يدعم العربية؟

نعم. يدعم أسئلة عربية، refusal عربي، و exam generation بالعربية.

سؤال 33: هل يدعم الإنجليزية؟

نعم. نفس ال APIs تدعم العربية والإنجليزية.

سؤال 34: كيف يتصل الفرونت بالباك؟

عن طريق HTTP requests لل FastAPI. الملفات ترفع multipart، والأسئلة ترسل JSON، وال token يرسل Bearer token.

سؤال 35: هل voice recognition موجود؟

من الفرونت باستخدام Web Speech API حسب دعم المتصفح. هذا يحول الصوت إلى نص ثم يرسل النص إلى Ask API. يمكن دعم voice input

سؤال 36: هل الصوت جزء من الباك؟

حاليا هو feature في الفرونت. مستقبلا يمكن إضافة backend speech-to-text مثل Whisper API.

سؤال 37: لماذا Docker؟

لأن Docker يضمن بيئة تشغيل ثابتة: build للفرونت، install للباك، dependencies، ثم تشغيل FastAPI.

سؤال 38: لماذا Railway؟

managed database و logs و environment variables و Docker مناسب للنشر السريع، يدعم Railway.

سؤال 39: ما سبب مشكلة `uvicorn: command not found`؟

container أو start command يعمل في بيئة غير صحيحة. الحل هو Dockerfile صحيح يثبت requirements ويشغل uvicorn. عادة تحدث عندما لا يتم تثبيت dependencies داخل

سؤال 40: ما سبب مشكلة memory في السيرفر؟

، parsing، وبناء context. لذلك نحتاج low-memory mode وتقليل token budget وربما vector DB خارجي لاحقاً. الـ RAG يستهلك ذاكرة في OCR، indexes، embeddings،

سؤال 41: كيف عالجت memory؟

low memory وتقليل answer tokens وتخزين PDF لإعادة بناء indexes. مستقبلاً يمكن نقل vector storage لخدمة خارجية. استخدمنا إعدادات

سؤال 42: هل API keys آمنة؟

يجب وضعها في environment variables فقط وعدم رفعها على GitHub.

سؤال 43: ما معنى MVC في المشروع؟

للطلبات، schemas للعقود، services للمنطق، repositories للداتا، models للجدول، و RAG modules للذكاء الاصطناعي. لدينا structure قريب من MVC/Clean Architecture: controllers

سؤال 44: لماذا فصلتم الكود؟

لأن الفصل يجعل المشروع أسهل في الصيانة والاختبار والتطوير، ويظهر كود production وليس notebook فقط.

سؤال 45: ما أقوى نقطة في المشروع؟

و persistent sources، hallucination risk، source-only answers، exams، chat، upload، login، AI safety: APIs منظمة. أنه يجمع بين منتج حقيقي

سؤال 46: ما نقطة الضعف الحالية؟

ليس persistent vector DB كامل حتى الآن. لكنه يعاد بناؤه من PDFs المحفوظة. الخطوة القادمة هي pgvector أو Qdrant. الـ vector index

سؤال 47: ماذا ستطورون لاحقاً؟

Persistent vector database، background jobs، أقوى OCR، export exams، admin dashboard، evaluation dataset، automated hallucination tests، backend voice recognition، و roles للطلاب والدكاترة.

سؤال 48: كيف تقيمون جودة الإجابة؟

، refusal accuracy، hallucination risk، groundedness، citation correctness، retrieval relevance، واستهلاك الذاكرة. latency نقيم

سؤال 49: ما معنى refusal accuracy؟

أن يرفض النظام فقط عندما لا يوجد دليل، ويجب عندما يكون الدليل موجودا.

سؤال 50: ما معنى citation correctness؟

أن الصفحة أو الدليل المذكور يحتوي فعلا على معلومة تدعم الإجابة.

سؤال 51: هل النظام مناسب للطب؟

هو مناسب كمساعد تعليمي source-grounded، لكنه ليس بديلا للطبيب. لذلك عند نقص الدليل يوصي باستشارة طبيب.

سؤال 52: هل يمكن استخدامه في الجامعات؟

نعم، في شرح المحاضرات، أسئلة الطلاب، توليد امتحانات، ومراجعة محتوى من مصدر الدكتور نفسه.

سؤال 53: هل يدعم أكثر من PDF؟

نعم، upload API يقبل أكثر من PDF ويبني knowledge base للشات.

سؤال 54: ماذا يحدث إذا رفع المستخدم ملفا غير PDF؟

الباك يرفضه برسالة validation لأن المدعوم حاليا PDF فقط.

سؤال 55: هل يمكن لمستخدم رؤية ملفات مستخدم آخر؟

التصميم يربط المصادر بال user وال chat ويستخدم authentication لتقليل هذا الخطر.

سؤال 56: لماذا يوجد raw في response؟

للتطوير وال debugging. في نسخة production أكثر صرامة يمكن تقليله أو إخفاؤه عن المستخدم العادي.

سؤال 57: ما الفرق بين confidence و groundedness؟

PDF. يقيس تحديدا مدى دعم الإجابة بالأدلة من ال groundedness تقدير عام لجودة الإجابة، بينما confidence

سؤال 58: هل يمكن تلخيص PDF؟

نعم، بشرط أن يكون التلخيص من محتوى ال PDF وليس من معرفة خارجية.

سؤال 59: هل يتعامل مع الجداول والرسوم؟

يوجد دعم pipeline للجداول والعناصر البصرية، لكن الجودة تعتمد على جودة ال PDF وال OCR والاستخراج.

سؤال 60: ماذا لو PDF scanned؟

يحتاج OCR. المشروع يحتوي على dependencies ووظائف مرتبطة بـ OCR، لكن جودة النتيجة تعتمد على وضوح المسح.

سؤال 61: ما أهم جملة تختتم بها العرض؟

لا يخترع الإجابات، بل يسترجع الأدلة، يتحقق منها، يقيس المخاطر، ويرفض عندما لا يكون المصدر كافياً DocuMind AI مبني على مبدأ أن المصدر المرفوع هو الحقيقة. الإيجنت

18. نقاط قوة يجب ذكرها في العرض

- المشروع ليس chatbot عادي.
- الـ Agent controlled وآمن.
- الـ PDF هو source of truth.
- الـ API منظم باستخدام Pydantic.
- يوجد hallucination risk و groundedness score.
- يوجد exam generation للدكاترة والجامعات.
- يوجد persistent source storage بعد restart.
- الباك منظم كـ production architecture.

19. إجابة جاهزة عن القيود

background، persistent vector database أقوى، اختبارات تقييم أكثر، monitoring، وتصدير الامتحانات كـ PDF أو Word. قوي كنموذج إنتاجي أولي و Hackathon project، لكن يمكن تطويره أكثر. أهم الخطوات القادمة هي OCR، indexing، المشروع

20. الخلاصة

هذا يجعله مناسباً للتعليم، الجامعات، والمجالات الحساسة التي تحتاج دقة ومصدر واضح PDF يحول ملفات DocuMind AI قوة المشروع ليست فقط في الإجابة، بل في أن الإجابة قابلة للتتبع والتحقق، وأن النظام يرفض عندما لا يوجد دليل كاف. إلى مصدر تفاعلي موثوق.